Software Engineering Lab

Lab 21

Transactional Outbox Pattern

Atomic Database Writes Meet Eventual Message Delivery



Lukman / lukman-ss

Dual-Write Problem: Ketidakonsistenan Senyap

Database commit sukses, tetapi broker publish gagal — order ada, event hilang.

SILENT EVENT LOSS

Poin 1

Database commit berhasil menyimpan order.

SILENT EVENT LOSS

Poin 2

Broker publish gagal karena jaringan down atau timeout.

SILENT EVENT LOSS

Poin 3

Order persisten, tetapi event OrderCreated tidak pernah dikirim ke downstream.

COUPLED OPERATIONS

Poin 1

Business logic langsung memanggil broker.publish().

COUPLED OPERATIONS

Poin 2

Hard to test, hard to retry, hard to recover.

COUPLED OPERATIONS

Poin 3

Kegagalan broker menggagalkan seluruh flow pembuatan order.

MANUAL RECONCILIATION

Poin 1

Perlu audit log untuk mendeteksi order tanpa event.

MANUAL RECONCILIATION

Poin 2

Script repair manual untuk replay event yang hilang.

MANUAL RECONCIL

Poin 3

Biaya operasion tinggi dan rentan hu error.

Dual-write problem muncul ketika aplikasi mencoba menulis ke database dan message broker secara terpisah dalam satu flow bisnis. Transaksi database hanya mengontrol operasi di dalam database itu sendiri; ia tidak dapat melakukan ROLLBACK pada sistem eksternal seperti Kafka atau RabbitMQ. Akibatnya, jika commit database sukses tetapi publish ke broker gagal, sistem masuk ke keadaan inkonsisten: order tersimpan tetapi event tidak pernah dikirim. Kondisi ini sulit dideteksi secara real-time dan memerlukan mekanisme rekonsiliasi manual yang kompleks.

Separate writes to independent systems cannot be atomic; decouple them with an outbox.

Mengapa Pendekatan Naif Gagal?

Kode dual-write terlihat sederhana, tetapi tidak memiliki jaminan atomisitas lintas sistem.

```
func CreateOrderDualWriteNaive(broker, orderID, amount) error {
    tx := db.BeginTx()
    tx.SaveOrder(order)
    tx.Commit()          // DB berhasil
    broker.Publish(msg)  // Broker gagal → inkonsisten
}
```

Implementasi naive dual-write tampaknya intuitif: simpan order ke database, lalu publish event ke broker. Namun, karena kedua operasi berada di luar boundary transaksi yang sama, kegagalan publish tidak dapat di-rollback dari sisi database. Order tetap tersimpan sementara event hilang selamanya. Pendekatan ini juga memperkenalkan coupling yang kuat antara service pemesanan dan broker, sehingga setiap kegagalan broker akan menggagalkan seluruh flow bisnis.

KEGAGALAN YG TERJADI

Order persisten di DB, tetapi event tidak sampai ke broker.
Tanpa outbox, tidak ada jejak pesan yang tertinggal untuk di-retry.
Reliable delivery must be decoupled dari business transaction.

Naive dual-write lacks cross-system atomicity; the outbox pattern restores it by staging events.

Mental Model: Transactional Outbox

Tulis entitas bisnis + event ke tabel outbox dalam satu transaksi database.

```
type OutboxMessage struct {
    ID        string
    EventType string
    Payload   string
    Status    MessageStatus  // PENDING, PROCESSED
    CreatedAt time.Time
}
```

Mental model inti dari pola Transactional Outbox adalah memindahkan penulisan event ke dalam boundary transaksi yang sama dengan penulisan entitas bisnis. Alih-alih melakukan dua write terpisah, aplikasi menulis order dan outbox message secara bersamaan dalam satu transaksi database. Setelah commit berhasil, kedua rekaman tersebut persisten. Proses asynchronous yang disebut message relay kemudian secara periodik polling record PENDING dan mengirimkannya ke broker. Jika relay gagal, record tetap PENDING dan akan dicoba lagi pada siklus berikutnya.

PRINSIP INTI

Atomic write: order + outbox message dalam satu Tx.
Decoupled relay: polling worker membaca PENDING records.
At-least-once delivery: retry hingga publish sukses.
Idempotent consumer: deduplication via event ID.

Commit business entity and its event together; let a relay eventually deliver.

Alur Eksekusi: Dari Transaksi Hingga Consumer

Pipeline端到端: Service → DB Tx → Relay → Broker → Consumer.

EKSEKUSI PIPELINE

```
Client → OrderService.CreateOrderWithOutbox
Service → DB.BeginTx() → SaveOrder + SaveOutbox (staged)
Service → DB.Commit() → flushed atomically
Relay (ticker) → GetPendingOutbox() → [PENDING]
Relay → Broker.Publish(msg) → success?
Relay → MarkOutboxProcessed(msg.ID) → PROCESSED
Consumer.Handle(msg) → idempotent check → accepted
```

Pipeline eksekusi Transactional Outbox terdiri dari lima tahap utama. Pertama, service menerima request pembuatan order dan memulai transaksi database. Kedua, service menyimpan order dan outbox message secara staged di dalam transaksi. Ketiga, service melakukan commit yang secara atomik flush kedua staged record ke database. Keempat, relay worker yang berjalan periodik polling record PENDING dan mencoba publish ke broker. Kelima, consumer menerima pesan dan melakukan idempotent check sebelum memproses lebih lanjut.

Atomic commit ensures both entity and event survive together; relay ensures eventual delivery.

internal/outbox/db.go:112–129 • internal/outbox/relay.go:50–67

Implementasi Atomic Outbox Write

Service.go: CreateOrderWithOutbox — staged save + atomic commit.

```
func (s *OrderService) CreateOrderWithOutbox(orderID string, customerID string, amount float64) error {
    tx := s.db.BeginTx()
    order := Order{ID: orderID, CustomerID: customerID, Amount: amount, Status: OrderStatusCreated}
    payload, _ := json.Marshal(order)
    msg := OutboxMessage{ID: fmt.Sprintf("evt-%s", orderID), EventType: "OrderCreated", Payload: string(payload), Status: MessageStatusPending, CreatedAt: time.Now()}
    tx.SaveOrder(order)
    tx.SaveOutbox(msg)
    return tx.Commit()
}
```

Stage both writes, commit once, guarantee both succeed or both fail.

Failure Path: Dual-Write Naive Menghasilkan Dangling Order

Order persisten, event hilang — bukti dari TestDualWriteProblem_Failure.

DEMO OUTPUT

```
=== Lab 21: Transactional Outbox Pattern Demo ===
[Scenario 1: The Dual-Write Problem]
Direct write failed: failed to publish to broker after DB commit: broker
unavailable
State Inconsistency: Order in DB = true, Broker Message Count = 0
```

ANALISIS

TestDualWriteProblem_Failure secara eksplisit mengassert ketidakonsistenan ini.
Order ada di DB, tetapi broker published count = 0.
Inconsistency terjadi karena broker publish dilakukan setelah commit.

Failure path ini membuktikan bagaimana dual-write naive menciptakan dangling reference: order yang persisten di database tanpa event yang sesuai di broker. Dalam demo, broker disimulasikan gagal melalui SetFailNext(true). Setelah commit database sukses, method CreateOrderDualWriteNaive mencoba publish ke broker yang gagal, menyebabkan error dikembalikan. Namun order sudah tersimpan permanen di database. Test Case TestDualWriteProblem_Failure mengassert kondisi ini: order ditemukan di DB, tetapi broker tidak menerima pesan apa pun.

When broker publish follows DB commit, failure leaves dangling orders without events.

Mekanisme vs Gejala: Dual-Write vs Outbox

Tabel perbandingan perilaku kedua pendekatan saat broker failure.

Aspek	Dual-Write Naive	Transactional Outbox
Write Boundary	Terpisah (DB commit dulu, lalu broker)	Sama (order + outbox dalam satu Tx)
Atomicity	Tidak atomik lintas sistem	Atomik untuk write pertama
Broker Failure Impact	Order persisten, event hilang	Order + event persisten, relay retry
Idempotency Requirement	Tidak diperlukan (karena konsisten)	Diperlukan (at-least-once delivery)
Operational Complexity	Rendah, tetapi risiko tinggi	Sedang, dengan relay dan cleanup

Tabel perbandingan ini menyoroti perbedaan fundamental antara pendekatan dual-write naive dan transactional outbox. Pada dual-write naive, boundary write terpisah antara database commit dan broker publish, sehingga tidak ada atomisitas lintas sistem. Ketika broker gagal setelah commit, order tetap persisten sedangkan event hilang. Sebaliknya, pada transactional outbox, kedua write dilakukan dalam satu transaksi database, menjamin atomisitas untuk tahap pertama. Broker failure hanya menunda delivery, bukan menghapus event, karena relay akan retry hingga sukses.

Outbox shifts the failure window from permanent loss to eventual delivery with retry.

3 Keputusan Desain Kritis

Urutan keputusan arsitektural yang harus diambil saat mengimplementasikan outbox.

KEPUTUSAN 1: ATOMIC BOUNDARY

Tentukan boundary transaksi yang mencakup write business entity + write outbox event.
Gunakan staging pattern: simpan sementara di memory, baru flush saat commit.

KEPUTUSAN 2: RELIABLE RELAY

Pilih strategi polling (Polling Publisher) atau log-tailing (CDC).
Relay harus retry hingga publish sukses dan hanya tandai PROCESSED setelah sukses.

KEPUTUSAN 3: IDEMPOTENT CONSUMER

Consumer harus memiliki deduplication mechanism berbasis event ID.
Handle(msg) mengembalikan true untuk pertama, false untuk duplikat.

Tiga keputusan desain kritis ini membentuk fondasi implementasi transactional outbox yang robust. Keputusan pertama menetapkan boundary transaksi yang atomik untuk write bisnis dan outbox. Keputusan kedua memilih mekanisme relay yang reliable dengan retry logic. Keputusan ketiga memastikan consumer dapat menangani duplicate delivery dengan aman melalui idempotency check. Ketiga keputusan ini saling bergantung dan harus diimplementasikan bersama untuk mencapai eventual consistency yang diinginkan.

Atomic boundary → reliable relay → idempotent consumer = eventual consistency.

Bukti Eksekusi: Terminal Output Demo

Output langsung dari go run ./cmd/demo yang menunjukkan dual-write failure dan outbox success.

DEMO TERMINAL

```
=== Lab 21: Transactional Outbox Pattern Demo ===

[Scenario 1: The Dual-Write Problem]
Direct write failed: failed to publish to broker after DB commit: broker unavailable
State Inconsistency: Order in DB = true, Broker Message Count = 0

[Scenario 2: Transactional Outbox Solution]
Broker received messages: 1
- Event ID: evt-order-outbox-success, Type: OrderCreated, Payload: {...}

[Scenario 3: At-Least-Once Delivery & Idempotent Consumer]
Consumer processing duplicate delivery: accepted=false
```

Terminal output di atas adalah bukti eksekusi langsung dari demo command yang menjalankan tiga skenario. Skenario 1 menunjukkan dual-write failure: order ada di DB tetapi broker message count = 0. Skenario 2 menunjukkan outbox success: satu pesan berhasil dipublikasikan ke broker setelah relay polling. Skenario 3 menunjukkan idempotent consumer: duplicate delivery ditolak dengan accepted=false, memastikan received count tetap 1. Output ini memverifikasi bahwa outbox pattern berhasil mengatasi dual-write problem.

Observable terminal output validates both failure and success paths.

YANG TIDAK DIBUKTIKAN

Batasan lab ini yang penting diketahui sebelum menerapkan ke production.

YANG TIDAK DIBUKTIKAN

Lab menggunakan in-memory DB, bukan PostgreSQL/MySQL real.
Hanya Polling Publisher yang diimplementasikan; CDC/log-tailing tidak ada.
Tidak ada dead-letter queue untuk poison payload non-retryable.
Skema OutboxMessage disederhanakan: tidak ada kolom aggregateid/aggregatetype.
Nilai SLA contoh (oldest-event age ~2s normal, 47+ menit abnormal) bersifat ilustratif.
Tidak ada benchmark throughput atau partisi skala tinggi.

Laboratorium ini memiliki beberapa batasan yang perlu diingat. Pertama, database yang digunakan adalah in-memory mock, bukan RDBMS produksi seperti PostgreSQL atau MySQL. Kedua, hanya strategi Polling Publisher yang diimplementasikan; pendekatan Transaction Log Tailing menggunakan CDC tools seperti Debezium tidak ditunjukkan. Ketiga, tidak ada dead-letter queue untuk menangani payload yang gagal berulang kali. Keempat, skema OutboxMessage disederhanakan tanpa kolom agregat yang biasanya ada di desain produksi. Kelima, nilai SLA yang disebutkan bersifat ilustratif dan harus disesuaikan dengan SLO masing-masing sistem.

Understand lab boundaries to map patterns correctly to production constraints.

Trade-off Arsitektur: Outbox Pattern

Konsistensi eventual vs kompleksitas operasional dan latency tambahan.

KEUNTUNGAN

Menghilangkan silent event loss dari dual-write naive.
Menjamin at-least-once delivery melalui retry mechanism.
Decoupling service pemesanan dari broker implementation.
Idempotent consumer mencegah double processing.

KEMALAHAN

Latency tambahan: event tidak langsung sampai ke broker, perlu polling interval.
Operational overhead: perlu monitor oldest-event age dan pending count.
Complexity tambahan: relay process, cleanup job, idempotency tracking.
Storage growth: outbox table bertambah hingga di-purge processed records.

Implementasi Transactional Outbox membawa trade-off yang perlu dipertimbangkan. Di sisi keuntungan, pola ini menghilangkan silent event loss yang sering terjadi pada dual-write naive dan menjamin at-least-once delivery melalui mekanisme retry. Pola ini juga mendekopling service pemesanan dari implementasi broker tertentu. Namun, ada kerugian yang signifikan: latency tambahan karena event tidak langsung sampai ke broker tetapi perlu menunggu polling interval. Selain itu, ada operational overhead berupa monitoring oldest-event age dan pending count, serta kompleksitas tambahan untuk relay process, cleanup job, dan idempotency tracking.

Trade consistency for operability: accept eventual delivery to prevent permanent loss.

Concurrency Safety: 10 Workers × 10 Orders = 100 Messages

Race detector bersih, consumer idempoten mencegah double processing.

```
func TestTransactionalOutbox_ConcurrentWrites(t *testing.T) {
    workers := 10
    ordersPerWorker := 10
    totalExpected := workers * ordersPerWorker  // 100
    // Goroutines menulis order + outbox secara konkuren
    // Relay polling dan publish
    // Assert: len(broker.GetPublished()) == 100
    // Assert: len(db.GetPendingOutbox()) == 0
}
```

Uji konkurensi membuktikan bahwa implementasi outbox pattern aman terhadap akses multi-goroutine. Dalam TestTransactionalOutbox_ConcurrentWrites, 10 worker goroutines masing-masing menghasilkan 10 order secara simultan, total 100 order. Race detector Go tidak melaporkan data race apapun. Setelah relay selesai memproses semua pending message, broker menerima tepat 100 pesan dan tidak ada yang tersisa di pending state. Test lain, TestTransactionalOutbox_ConcurrentConsumers, memverifikasi bahwa 10 worker yang mengirimkan pesan dengan ID sama hanya menghasilkan 20 unique processed message, membuktikan idempotency kerja dengan baik di bawah konkurensi.

HASIL

Race detector: PASS
Total published: 100
Pending after processing: 0
Consumer concurrent test: 20 unique processed, duplicates rejected.

Mutex-protected state and deterministic ID tracking ensure safe concurrent processing.

tests/outbox_test.go:142–194 • tests/outbox_test.go:266–293

Before vs After: Transformasi Arsitektur

Perbandingan visual keadaan sebelum dan sesudah menerapkan outbox pattern.

BEFORE (DUAL-WRITE NAIVE)

Struktur: Service → DB.Commit → Broker.Publish
Keadaan: Order persisten, event mungkin hilang
Kegagalan: Broker down = inconsistent state
Testing: Sulit mengassert consistency lintas sistem
Ops: Perlu script rekonsiliasi manual

AFTER (TRANSACTIONAL OUTBOX)

Struktur: Service → DB.Tx(Order+Outbox) → Relay → Broker
Keadaan: Order + event persisten atomik, delivery eventual
Kegagalan: Broker down = retry, tidak ada yang hilang
Testing: Unit test atomis + integration test end-to-end
Ops: Monitor relay health dan oldest-event age

Transformasi dari dual-write naive ke transactional outbox mengubah arsitektur secara fundamental. Sebelum menerapkan pola ini, service menulis ke database dan broker secara berurutan tanpa jaminan atomisitas. Jika broker gagal, order tetap tersimpan tetapi event hilang, menciptakan inconsistent state yang sulit dideteksi. Setelah menerapkan pola ini, kedua write dilakukan dalam satu transaksi database, sehingga baik order maupun event persisten bersama. Broker failure hanya menunda delivery, bukan menghapus event, karena relay akan terus retry hingga sukses.

Before: coupled and fragile. After: decoupled and resilient.

Test 1: Happy Path – Atomic Write + Relay Dispatch

TestTransactionalOutbox_HappyPath membuktikan全流程 sukses.

```
func TestTransactionalOutbox_HappyPath(t *testing.T) {
    db := outbox.NewDB()
    broker := outbox.NewMockBroker()
    relay := outbox.NewRelay(db, broker, 10*time.Millisecond)
    service := outbox.NewOrderService(db)
    consumer := outbox.NewConsumer()
    relay.Start()
    err := service.CreateOrderWithOutbox("o-1", "c-1", 100.50)
    assert.NoError(t, err)
    time.Sleep(50 * time.Millisecond)
    published := broker.GetPublished()
    assert.Len(t, published, 1)
    assert.Equal(t, "OrderCreated", published[0].EventType)
    msg, _ := db.GetOutbox("evt-o-1")
    assert.Equal(t, outbox.MessageStatusProcessed, msg.Status)
}
```

Test ini membuktikan happy path lengkap dari transactional outbox. Pertama, service membuat order dengan ID o-1 secara atomik bersama outbox message. Setelah transaksi commit, relay yang berjalan dengan interval 10ms akan polling dan menemukan message PENDING. Relay kemudian publish ke broker dan menandai message sebagai PROCESSED. Consumer menerima pesan dan memprosesnya. Test ini mengassert bahwa broker menerima tepat satu pesan dengan EventType OrderCreated, dan status outbox message berubah menjadi PROCESSED. Semua assertion lolos, membuktikan alur端到端 berfungsi dengan benar.

Happy path: atomic write → relay poll → broker publish → consumer accept.

tests/outbox_test.go:12–60

Test 2: Rollback – Nothing Persists

TestTransactionalOutbox_Rollback membuktikan staged mutation dibuang sempurna.

```
func TestTransactionalOutbox_Rollback(t *testing.T) {
    db := outbox.NewDB()
    tx := db.BeginTx()
    tx.SaveOrder(outbox.Order{ID: "o-2", Amount: 50})
    tx.SaveOutbox(outbox.OutboxMessage{ID: "evt-o-2", Status: outbox.MessageStatusPending})
    tx.Rollback()
    _, ok := db.GetOrder("o-2")
    assert.False(t, ok, "order should not exist")
    _, ok = db.GetOutbox("evt-o-2")
    assert.False(t, ok, "outbox should not exist")
    time.Sleep(30 * time.Millisecond)
    assert.Len(t, broker.GetPublished(), 0, "no messages to broker")
}
```

Test rollback membuktikan bahwa staged mutation dibuang sepenuhnya ketika transaksi di-rollback. Method SaveOrder dan SaveOutbox hanya menaruh data di map stagedOrders dan stagedOutbox, belum menulis ke database utama. Ketika Rollback dipanggil, kedua map staged tersebut dibuang tanpa pernah menyentuh database. Test mengassert bahwa order dan outbox message tidak dapat diambil dari database setelah rollback. Selain itu, setelah menunggu relay poll, broker tidak menerima pesan apa pun, membuktikan bahwa rollback mencegah event dispatch sama sekali.

Rollback discards staged writes; no persistence, no dispatch.

tests/outbox_test.go:62–91

Test 3: Idempotency – Duplicate Delivery Rejected

TestTransactionalOutbox_Idempotency_DuplicateDelivery membuktikan consumer safe.

```
func TestTransactionalOutbox_Idempotency_DuplicateDelivery(t *testing.T) {
    consumer := outbox.NewConsumer()
    msg := outbox.OutboxMessage{ID: "evt-duplicate", EventType: "OrderCreated", Payload: "data"}
    p1 := consumer.Handle(msg)
    assert.True(t, p1, "first delivery should be accepted")
    p2 := consumer.Handle(msg)
    assert.False(t, p2, "duplicate delivery should be rejected")
    assert.Equal(t, 1, consumer.GetReceivedCount(), "count stays 1")
}
```

Test idempotensi membuktikan bahwa consumer dapat dengan aman menangani duplicate delivery. Pertama, Handle dipanggil dengan pesan unik dan mengembalikan true (accepted). Kedua, Handle dipanggil lagi dengan pesan yang sama dan mengembalikan false (rejected). Test mengassert bahwa received count tetap 1, membuktikan bahwa duplicate tidak menambah state konsumen. Skenario ini mensimulasikan kasus di mana relay crash setelah publish tetapi sebelum menandai message sebagai PROCESSED, menyebabkan pesan yang sama dipublish ulang oleh relay pada siklus berikutnya.

Idempotent consumers deduplicate by event ID; second delivery is ignored.

tests/outbox_test.go:93–116

Test 4: Relay Retry – Broker Failure Tolerated

TestTransactionalOutbox_RelayRetryAfterBrokerFailure membuktikan retry logic.

```
func TestTransactionalOutbox_RelayRetryAfterBrokerFailure(t *testing.T) {
    db := outbox.NewDB()
    broker := outbox.NewMockBroker()
    relay := outbox.NewRelay(db, broker, 10*time.Millisecond)
    service := outbox.NewOrderService(db)
    broker.SetFailNext(true)  // first publish fails
    service.CreateOrderWithOutbox("o-retry", "c-retry", 55.0)
    relay.Start()
    time.Sleep(25 * time.Millisecond)  // first poll fails
    msg, _ := db.GetOutbox("evt-o-retry")
    assert.Equal(t, outbox.MessageStatusPending, msg.Status)
    time.Sleep(50 * time.Millisecond)  // second poll succeeds
    published := broker.GetPublished()
    assert.Len(t, published, 1)
    msg, _ = db.GetOutbox("evt-o-retry")
    assert.Equal(t, outbox.MessageStatusProcessed, msg.Status)
}
```

Test relay retry membuktikan mekanisme ketahanan terhadap kegagalan broker. Pertama, broker diatur untuk gagal pada publish berikutnya melalui SetFailNext(true). Service membuat order dengan outbox, kemudian relay dimulai. Pada poll pertama, publish gagal karena broker dalam mode fail, sehingga message tetap berstatus PENDING. Pada poll berikutnya (setelah interval ticker), broker kembali normal, publish sukses, dan message ditandai PROCESSED. Test ini mengassert bahwa setelah retry, broker menerima pesan dan status outbox berubah menjadi PROCESSED.

Relay retry converts transient broker failures into eventual delivery.

tests/outbox_test.go:219-264

Test 5: Purge Processed – Memory Leak Prevention

TestTransactionalOutbox_PurgeProcessed membuktikan cleanup mechanism bekerja.

```
func TestTransactionalOutbox_PurgeProcessed(t *testing.T) {
    db := outbox.NewDB()
    tx := db.BeginTx()
    tx.SaveOutbox(outbox.OutboxMessage{ID: "m1", Status: outbox.MessageStatusPending})
    tx.SaveOutbox(outbox.OutboxMessage{ID: "m2", Status: outbox.MessageStatusProcessed})
    tx.Commit()
    purged := db.PurgeProcessedOutbox()
    assert.Equal(t, 1, purged)
    _, ok := db.GetOutbox("m1")
    assert.True(t, ok, "pending should remain")
    _, ok = db.GetOutbox("m2")
    assert.False(t, ok, "processed should be purged")
}
```

Test purge processed membuktikan mekanisme pembersihan tabel outbox untuk mencegah memory leak. Database awalnya berisi dua message: m1 dengan status PENDING dan m2 dengan status PROCESSED. Method PurgeProcessedOutbox akan menghapus semua record dengan status PROCESSED dan mengembalikan jumlah record yang dihapus. Test mengassert bahwa purged count = 1, message PENDING masih ada, dan message PROCESSED sudah dihapus. Tanpa cleanup mechanism seperti ini, tabel outbox akan tumbuh tak terbatas karena record PROCESSED tidak pernah dihapus.

Purge processed records to prevent unbounded outbox table growth.

tests/outbox_test.go:196–217

Checklist Implementasi Outbox Pattern

Langkah-langkah wajib saat menerapkan pola ini ke production.

- ✓ ☐ Buat tabel outbox_events dengan kolom: id, aggregate_type, aggregate_id, type, payload, status, created_at
- ✓ ☐ Pastikan business transaction menyertakan INSERT ke outbox_events
- ✓ ☐ Implementasikan polling relay dengan interval yang sesuai (misal 1–5 detik)
- ✓ ☐ Relay hanya menandai PROCESSED setelah broker ACK
- ✓ ☐ Consumer mengimplementasikan idempotency check berdasarkan event ID
- ✓ ☐ Tambahkan scheduled job untuk purge processed records
- ✓ ☐ Monitor oldest-event age dan pending count untuk deteksi degradasi
- ✓ ☐ Siapkan dead-letter queue untuk poison payload (opsional)

Implementation requires atomic writes, reliable relay, idempotent consumers, and cleanup.

Anti-Pattern: Apa yang Harus Dihindari

Kesalahan umum yang membuat outbox pattern menjadi tidak efektif.

ANTI-PATTERN 1

Menulis ke broker sebelum commit database.
Menggunakan ID event yang tidak unik (misal timestamp saja).
Tidak memiliki mekanisme retry pada relay.

ANTI-PATTERN 2

Consumer tanpa idempotency check.
Tidak ada purge untuk processed records.
Relay polling interval terlalu lama (>1 menit) sehingga latency tinggi.

CARA REPRODUKSI GAGAL

Jalankan TestDualWriteProblem_Failure untuk melihat inconsistency.
Matikan broker selama relay berjalan, perhatikan pending accumulation.
Hapus idempotency check dari consumer, maka duplicate akan diproses ganda.

Anti-pattern dalam implementasi outbox pattern dapat merusak seluruh jaminan konsistensi yang dimaksudkan. Menulis ke broker sebelum commit database mengembalikan ke keadaan dual-write naive. Menggunakan ID event yang tidak unik menyebabkan deduplication gagal. Consumer tanpa idempotency check akan memproses duplicate message secara ganda. Tidak ada purge untuk processed records akan menyebabkan tabel outbox tumbuh tak terbatas. Relly dengan polling interval terlalu lama akan meningkatkan latency delivery secara signifikan.

Avoid coupling, non-unique IDs, and missing idempotency; they undo the pattern's benefits.

internal/outbox/service.go:55-90 • tests/outbox_test.go:118-140

Takeaway Praktis & Command Block

Cara mereproduksi lab dan menjalankan verifikasi.

```
cd labs/21-outbox-pattern
go test -v ./...
go test -race ./...
go run ./cmd/demo
```

PRAKTIK TERBAIK

Gunakan event ID yang stabil (UUID berbasis aggregate ID).
Monitor oldest-event age sebagai sinyal degradasi relay/broker.
Pertahankan pending count di bawah threshold tertentu.
Implementasikan dead-letter queue untuk payload yang gagal berulang.

Takeaway praktis dari lab ini adalah pemahaman tentang cara mereproduksi dan memverifikasi implementasi outbox pattern. Command go test -v ./... menjalankan seluruh suite test dengan verbose output. Command go test -race ./... menjalankan race detector untuk memastikan thread safety. Command go run ./cmd/demo menjalankan demo interaktif yang menampilkan tiga skenario. Praktik terbaik meliputi penggunaan event ID yang stabil berdasarkan aggregate ID, monitoring oldest-event age sebagai sinyal degradasi, mempertahankan pending count di bawah threshold, dan mengimplementasikan dead-letter queue untuk payload yang gagal berulang.

Reproduce, measure, monitor — then trust eventual consistency.

Prinsip Arsitektural Jangka Panjang

Filosofi desain yang melampaui pola outbox spesifik.

PRINSIP 1

Eventual consistency lebih baik daripada strong consistency yang mahal.
Decouple write boundary dari delivery boundary.
At-least-once delivery lebih aman daripada at-most-once.

PRINSIP 2

Idempotency adalah keharusan, bukan optional.
Observable metrics (oldest-event age, pending count) wajib diimplementasikan.
Cleanup mechanism harus berjalan scheduled, bukan manual.

Prinsip arsitektural jangka panjang ini melampaui implementasi spesifik transactional outbox. Pertama, eventual consistency seringkali lebih praktis daripada strong consistency yang memerlukan koordinasi terdistribusi mahal. Kedua, memisahkan boundary write dari boundary delivery memungkinkan masing-masing bagian dioptimalkan secara independen. Ketiga, at-least-once delivery lebih aman daripada at-most-once karena mencegah kehilangan data. Keempat, idempotency bukan fitur optional melainkan keharusan karena retry adalah hal yang wajar. Kelima, observable metrics seperti oldest-event age dan pending count harus diimplementasikan untuk deteksi dini degradasi.

Design for failure, measure relentlessly, accept eventual consistency.

Konsistensi tercapai, event tak pernah hilang.

Atomic outbox writes + eventual relay delivery = no silent event loss.

Software Engineering Lab 21 – Transactional Outbox Pattern

internal/outbox/db.go
internal/outbox/service.go
internal/outbox/relay.go
internal/outbox/consumer.go
internal/outbox/broker.go
tests/outbox_test.go
cmd/demo/main.go

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/21-outbox-pattern`

Lukman / lukman-ss

internal/outbox/db.go
internal/outbox/service.go
internal/outbox/relay.go
internal/outbox/consumer.go
internal/outbox/broker.go
tests/outbox_test.go
cmd/demo/main.go
Sumber: commit b2d32740
Test passing: 8/8 test cases
Race detector: clean
Demo: all 3 scenarios pass